Metaheurísticas

Aula 3 — Hill Climbing com Reinício e Simulated Annealing

Raoni F. S. Teixeira · 1s/2026

Objetivos desta aula

Ao final desta aula, o aluno deve ser capaz de:

1. Implementar e analisar o Hill Climbing com reinício aleatório, identificando suas limitações estruturais;
2. Derivar o critério de aceitação de Metropolis a partir de princípios termodinâmicos;
3. Comparar e justificar diferentes esquemas de resfriamento (linear, geométrico, logarítmico e adaptativo), analisando o trade-off convergência vs. qualidade;
4. Aplicar Simulated Annealing ao Problema do Caixeiro-Viajante como estudo de caso;
5. Calibrar os parâmetros do SA com base em critérios analíticos.

1 Metaheurísticas de Trajetória: visão geral

Na Aula 2, estabelecemos que toda metaheurística enfrenta o mesmo dilema: equilibrar exploração (buscar novas regiões) com explotação (refinar o que já foi encontrado). Os algoritmos estudados nesta parte do curso pertencem à família das **metaheurísticas de trajetória**: operam sobre uma única solução candidata que se move pelo espaço de busca, gerando uma sequência — uma trajetória — de estados visitados.

A trajetória pode ser estritamente descendente (**Hill Climbing**), controladamente não-monotônica (**Simulated Annealing**) ou guiada por memória explícita (Busca Tabu, Aula 4). Cada estratégia endereça, a seu modo, o problema fundamental identificado na Aula 1: como escapar de mínimos locais sem avaliar exaustivamente o espaço de busca.

Nesta aula, cobrimos os dois primeiros algoritmos. O Hill Climbing serve como algoritmo base cujas limitações motivam diretamente cada extensão introduzida pelo SA.

2 Hill Climbing com Reinício Aleatório

2.1 O algoritmo básico

O **Hill Climbing** (HC) é o método de trajetória mais simples: a cada iteração, examina-se a vizinhança da solução corrente e move-se para o melhor vizinho que seja melhor que o estado atual. Se nenhum vizinho melhorar a solução, o algoritmo para — atingiu um **ótimo local**.

Algoritmo — Hill Climbing com Reinício Aleatório

```
HC-Reinicio(f, Vizinhanca, max_reinicio, max_iter):
  melhor_global ← NIL
  para r = 1 até max_reinicio:
    s ← solucao_aleatoria()
    para it = 1 até max_iter:
      s_viz ← argmin { f(v) : v ∈ Vizinhanca(s) }
      se f(s_viz) < f(s):
        s ← s_viz
      senão:
        break   // ótimo local atingido
    se melhor_global = NIL ou f(s) < f(melhor_global):
      melhor_global ← s
  retorna melhor_global
```

O loop externo (reinícios) é a modificação crítica em relação ao HC básico. Cada reinício parte de um ponto aleatório distinto, aumentando a probabilidade de que ao menos um deles esteja na bacia de atração do ótimo global.

2.2 Análise de complexidade e cobertura

Seja $\mathcal{F}$ o espaço viável com $|\mathcal{F}| = N$ estados. A probabilidade de que o ótimo global **não** seja encontrado em nenhum dos R reinícios é:

$$P(\text{falha}) = 1 - \frac{|\mathcal{B}^*|}{N}^R$$

onde $|\mathcal{B}^*|$ é o tamanho da bacia de atração do ótimo global. Para garantir $P(\text{falha}) \leq \delta$, o número mínimo de reinícios necessários é:

$$R \geq \frac{\ln \delta}{\ln\left(1 - |\mathcal{B}^* \frac{|}{N}\right)}$$

🔁 **Fio Condutor — Problema da Mochila**

Problema da Mochila — cobertura por reinício. Para a instância da Aula 1 ($n = 20$ itens, $W = \lfloor 0.5 \sum w_i \rfloor$), o espaço tem $2^{\{20\}} \approx 10^6$ soluções binárias. Sob vizinhança 1-flip, cada reinício do HC percorre no máximo $n = 20$ passos antes de atingir um ótimo local. Se a bacia de atração do ótimo global cobre apenas 1% do espaço ($|\mathcal{B}^* \frac{|}{N} = 0.01$), para $\delta = 0.05$:

$R \geq \frac{\ln(0.05)}{\ln(0.99)} \approx 299$ reinícios.

Com $n = 20$ avaliações por reinício, isso exige ≈ 6000 avaliações de f — barato para o Knapsack ($O(n)$ por avaliação), mas o custo escala com 2^n para instâncias maiores. Para $n = 50$, a bacia do ótimo global pode cobrir uma fração exponencialmente menor do espaço, tornando o número de reinícios necessários astronomicamente grande.

2.3 Limitações fundamentais

Memorylessness. Cada reinício ignora toda informação acumulada nas tentativas anteriores. O algoritmo não aprende quais regiões já foram exploradas — problema que a Busca Tabu resolve explicitamente (Aula 4).

Sensibilidade à definição de vizinhança. A qualidade do ótimo local encontrado depende criticamente de como $\mathcal{N}(s)$ é definida. Uma vizinhança pequena converge rápido mas pode prender o algoritmo em ótimos locais rassos. Uma vizinhança grande melhora a qualidade mas aumenta o custo por iteração.

Sem mecanismo de escape. Uma vez em um ótimo local, o HC básico não tem como sair — exceto pelo reinício. O SA resolve esse problema de forma elegante, como veremos a seguir.

3 Simulated Annealing

3.1 Motivação termodinâmica

O **Simulated Annealing** (SA) foi proposto por Kirkpatrick, Gelatt e Vecchi em 1983, inspirado no processo físico de **recozimento** (*annealing*): um metal aquecido a alta temperatura tem seus átomos em estado de alta energia e grande mobilidade. O resfriamento lento permite que o sistema encontre configurações de energia mínima global (cristal perfeito); o resfriamento brusco "congela" defeitos — análogos a ótimos locais.

A correspondência com otimização é direta: estados do sistema físico $\leftrightarrow$ soluções do problema; energia $\leftrightarrow$ função objetivo; temperatura $\leftrightarrow$ parâmetro de controle que governa a aceitação de pioras.

3.2 O critério de Metropolis

O coração do SA é o critério de aceitação de **Metropolis** (1953), originalmente desenvolvido para simulação de sistemas termodinâmicos:

Definição — Critério de Metropolis
Dado o estado corrente s e um candidato s' gerado aleatoriamente na vizinhança de s, defina $\Delta f = f(s') - f(s)$. A transição $s \to s'$ é aceita com probabilidade:

$$P(\text{aceitar} \mid \Delta f, T) = \begin{cases} 1 & \text{se } \Delta f < 0 \quad \text{(melhora sempre aceita)} \\ e^{-\Delta f / T} & \text{se } \Delta f \geq 0 \quad \text{(piora aceita com probabilidade decrescente)} \end{cases}$$

onde $T > 0$ é a **temperatura** atual do algoritmo.

Note que P é uma função de dois argumentos: a magnitude da piora Δf e a temperatura T.

- Quando $T \to \infty$: $P \to 1$ para qualquer Δf — o algoritmo aceita qualquer movimento (busca aleatória pura).
- Quando $T \to 0$: $P \to 0$ para $\Delta f > 0$ — o algoritmo se comporta como Hill Climbing.
- Para T intermediário: pioras pequenas (Δf pequeno) têm probabilidade razoável de aceitação; pioras grandes são raramente aceitas.

Essa propriedade garante que o SA possa **escapar** de ótimos locais — especialmente no início da execução, quando T é alto — enquanto **refina** progressivamente a solução à medida que T diminui.

Observação
A analogia física vai além da inspiração: sob certas condições (resfriamento **suficientemente lento**), o SA converge com probabilidade 1 para o ótimo global. Esse resultado, provado por Geman & Geman (1984), exige que $T_k \geq \frac{C}{\ln(1+k)}$ para alguma constante C — uma condição praticamente

impossível de satisfazer em tempo finito. Na prática, usamos esquemas de resfriamento mais rápidos e abrimos mão da garantia teórica.

3.3 Algoritmo completo

Algoritmo — Simulated Annealing

```
SA(f, Vizinhanca, T0, alpha, T_min, max_iter_T):
  s   ← solucao_aleatoria()
  T   ← T0
  s*  ← s          // guarda melhor solução encontrada
  enquanto T > T_min:
    para it = 1 até max_iter_T:
      s' ← vizinho_aleatorio(s, Vizinhanca)
      ΔF ← f(s') - f(s)
      se ΔF < 0:
        s ← s'
        se f(s) < f(s*): s* ← s
      senão:
        se uniforme(0,1) < exp(-ΔF / T):
          s ← s'    // aceita piora
    T ← resfriar(T, alpha)   // atualiza temperatura
  retorna s*
```

Dois detalhes importantes na implementação:

1. **Guardar o melhor global s^*.** O SA pode aceitar pioras e se afastar da melhor solução encontrada. É essencial registrar s^* separadamente e retorná-lo ao final.
2. **Iterações por temperatura** (`max_iter_T`). Cada nível de temperatura deve ser amostrado suficientemente antes de resfriar — analogia ao **equilíbrio térmico** da física. Valores típicos: $L = c \cdot n$ onde n é o tamanho da vizinhança e $c \in [1, 10]$.

4 Esquemas de Resfriamento

O **esquema de resfriamento** (*cooling schedule*) define como T diminui ao longo das iterações. É o parâmetro de projeto mais crítico do SA: resfriamento rápido demais congela o algoritmo em ótimos locais; lento demais desperdiça avaliações. A tabela a seguir compara os principais esquemas.

Esquema	Fórmula	Parâmetros	Características
Geométrico	$T_{k+1} = \alpha \cdot T_k$	$\alpha \in (0.90, 0.99)$	Mais usado na prática; fácil de implementar; boa relação velocidade/qualidade
Linear	$T_k = T_0 - k \cdot \Delta T$	$\Delta T = \frac{T_0 - T_f}{K}$	Simples; temperatura cai uniformemente; ineficiente — muito tempo em temperaturas baixas irrelevantes
Logarítmico	$T_k = \frac{T_0}{\ln(1+k)}$	—	Convergência teórica garantida; extremamente lento; impraticável para instâncias grandes
Adaptativo	$T \leftarrow T \cdot e^{-\frac{\overline{\Delta f}}{\sigma_{\Delta f}}}$	janela W	Ajusta T com base na variância das diferenças de custo observadas; robusto a mudanças de escala de f

Tabela 1: Comparação de esquemas de resfriamento para Simulated Annealing.

4.1 Resfriamento geométrico: análise detalhada

O esquema geométrico $T_{k+1} = \alpha \cdot T_k$ produz a sequência $T_0, \alpha T_0, \alpha^2 T_0, \ldots$. Após K níveis de temperatura, tem-se $T_K = \alpha^K T_0$. Para atingir T_f a partir de T_0:

$$K = \frac{\ln\left(\frac{T_f}{T_0}\right)}{\ln \alpha}$$

O número total de avaliações de f é $K \times L$, onde L é o número de iterações por nível de temperatura.

Como escolher α? A regra prática de Johnson et al. (1989) sugere que a taxa de aceitação no início deve ser $\approx 80\%$ e ao final $\approx 1\%$. Isso determina indiretamente T_0 e T_f; α controla a velocidade de transição entre esses extremos.

4.2 Temperatura inicial: método da taxa de aceitação

Uma técnica sistemática para definir T_0 parte de uma taxa de aceitação inicial desejada χ_0 (tipicamente 0.8):

1. Execute uma sequência de n_0 movimentos aleatórios; registre as pioras $\{\Delta f_i^+ : i = 1, \ldots, m^+\}$ com $\Delta f_i^+ > 0$.
2. Resolva $\chi_0 = \exp\left(-\frac{\overline{\Delta f^+}}{T_0}\right)$ para T_0:

$$T_0 = -\frac{\overline{\Delta f^+}}{\ln \chi_0}$$

onde $\overline{\Delta f^+} = \left(\frac{1}{m^+}\right) \sum_i \Delta f_i^+$ é a piora média observada.

> ⚠️ **Atenção**
> **Calibração é essencial, não opcional.** Um SA com T_0 mal escolhido pode ser indistinguível de busca aleatória (T_0 muito alto) ou de Hill Climbing (T_0 muito baixo). Sempre use o método da taxa de aceitação ou outro procedimento sistemático antes de reportar resultados.

4.3 Esquema adaptativo: SA com controle de variância

O esquema adaptativo de Huang et al. (1986) atualiza a temperatura baseando-se na variância das diferenças de custo observadas na janela atual:

$$T' = T \cdot \exp -0.7 \cdot \frac{T}{\sigma_{\Delta f}}$$

onde $\sigma_{\Delta f}$ é o desvio padrão de $\{\Delta f\}$ na janela de W iterações recentes. Esse esquema reduz T mais rápido quando a paisagem é suave (baixa variância) e mais devagar quando a paisagem é rugosa (alta variância) — adaptando-se automaticamente à estrutura local do problema.

5 Estudo de Caso: SA no Problema do Caixeiro-Viajante

O **Problema do Caixeiro-Viajante** (TSP — *Travelling Salesman Problem*) é um dos benchmarks mais estudados em otimização combinatória e um campo de testes natural para metaheurísticas de trajetória. Seu enunciado é simples: dadas n cidades com distâncias conhecidas, encontrar a rota que visita cada cidade exatamente uma vez e retorna à origem com distância total mínima.

O TSP é NP-difícil, sua paisagem de busca é fortemente multimodal, e o custo de avaliação de f é $O(n)$ — barato o suficiente para permitir orçamentos grandes e experimentos controlados sem infraestrutura especial.

> 🔁 **Fio Condutor — Problema da Mochila**
> O TSP complementa o Problema da Mochila no fio condutor do curso: enquanto o Knapsack é discreto/binário com uma restrição de capacidade, o TSP é discreto/permutacional sem restrições explícitas de viabilidade. Juntos, cobrem dois dos tipos mais comuns de estrutura combinatória. Nas próximas aulas, veremos como Busca Tabu, Algoritmos Genéticos e ACO atacam o TSP de formas distintas.

5.1 Formulação

Dadas n cidades com coordenadas (x_i, y_i), a distância euclidiana entre as cidades i e j é $d_{\{i,j\}} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$. Uma solução é uma permutação $\pi = (\pi_1, \pi_2, ..., \pi_n)$ das cidades. A função objetivo é:

$$f(\pi) = \sum_{k=1}^{n-1} d_{\pi_k, \pi_{k+1}} + d_{\pi_n, \pi_1}$$

O espaço de soluções tem $\frac{(n-1)!}{2}$ rotas distintas (descontando rotações e reflexões equivalentes): para $n = 20$ cidades, são $\approx 6 \times 10^{\{16\}}$ rotas — inviável por enumeração.

5.2 Vizinhança: operador 2-opt

O operador **2-opt** é a vizinhança padrão para o TSP: dado um tour π, um vizinho é obtido removendo dois arcos não-adjacentes e reconectando os segmentos resultantes na única forma alternativa possível.

Tour original	**Tour 2-opt**
$\ldots \to \pi_i \to \pi_{i+1} \to \ldots \to \pi_j \to \pi_{j+1} \to \ldots$	$\ldots \to \pi_i \to \pi_j \to \ldots \to \pi_{i+1} \to \pi_{j+1} \to \ldots$
Arcos removidos: (π_i, π_{i+1}) e (π_j, π_{j+1})	O segmento entre $i+1$ e j é invertido.

Para n cidades, a vizinhança 2-opt tem $\binom{n}{2} - n = \frac{n(n-3)}{2}$ vizinhos. Para $n = 20$, são 170 vizinhos por avaliação de vizinhança completa — custo gerenciável. Para $n = 100$, são 4850 vizinhos — ainda viável, mas o SA geralmente usa amostragem aleatória da vizinhança em vez de enumeração completa.

A variação de custo ao aplicar o movimento 2-opt nos arcos $(i, i+1)$ e $(j, j+1)$ é:

$$\Delta f = d_{\pi_i,\pi_j} + d_{\pi_{i+1},\pi_{j+1}} - d_{\pi_i,\pi_{i+1}} - d_{\pi_j,\pi_{j+1}}$$

Se $\Delta f < 0$, o movimento é uma melhora e sempre aceito. Se $\Delta f \geq 0$, o critério de Metropolis decide.

5.3 Calibração e experimento

Exemplo

TSP com $n = 20$ cidades geradas aleatoriamente (coordenadas uniformes em $[0, 100]^2$, semente 42).

Calibração de T_0: 500 movimentos 2-opt aleatórios sobre tours aleatórios produziram piora média $\overline{\Delta f^+} = 38.4$ km. Para $\chi_0 = 0.80$: $T_0 = -\frac{38.4}{\ln(0.80)} \approx 172 \quad \text{km}$

Parâmetros adotados: $\alpha = 0.95, T_{\min} = 0.1$ km, $L = 5 \cdot \binom{20}{2} = 950$ iterações/nível, totalizando $\approx 950 \times \frac{\ln\left(\frac{0.1}{172}\right)}{\ln(0.95)} \approx 146.000$ avaliações de f.

Método	$\overline{f^*}$ (km)	σ_{f^*}	Melhor	NFE médio
HC · 50 reinícios	892	41	823	18 500
SA (geométrico)	831	18	802	146 000
SA (adaptativo)	818	12	798	146 000

O SA encontrou tours consistentemente melhores que o HC, com desvio padrão $\approx 3 \times$ menor — reflexo do mecanismo de escape de Metropolis, que permite atravessar barreiras entre bacias de atração que o HC nunca ultrapassaria. O SA adaptativo obteve a menor variância, ajustando o resfriamento às irregularidades locais da paisagem. Teste de Wilcoxon entre HC e SA geométrico: $p < 0.001$, tamanho de efeito $r = 0.71$ (grande).

Diagnóstico pela curva de aceitação: a taxa de aceitação do SA caiu de 79% (iteração 1) para 8% (metade do orçamento) e 0.3% (final) — perfil saudável indicando transição gradual de exploração para explotação.

6 Análise do Comportamento do SA

6.1 Curva de temperatura vs. taxa de aceitação

A evolução da taxa de aceitação ao longo do resfriamento é um diagnóstico valioso:

- **Fase inicial** (T alto, aceitação $> 60\%$): o algoritmo explora amplamente — análogo à fase de busca aleatória.
- **Fase intermediária** (aceitação entre 10% e 60%): balanço exploração-explotação; o algoritmo se move para regiões promissoras enquanto ainda aceita pioras moderadas.
- **Fase final** (T baixo, aceitação $< 5\%$): refinamento local; o comportamento converge para Hill Climbing.

Se a taxa de aceitação cair abruptamente no início (antes de T atingir a fase intermediária), o resfriamento é muito rápido — reduza α ou aumente T_0.

6.2 Reaquecimento

Uma extensão prática do SA é o **reaquecimento** (*reheating*): quando a taxa de aceitação cai abaixo de um limiar (e.g., 1%) antes de atingir $T_{\min}$, a temperatura é restaurada para um valor intermediário ($\frac{T_0}{10}$, por exemplo) e o resfriamento recomeça. Isso permite múltiplos ciclos de exploração-refinamento sem aumentar o orçamento total.

> ⚠️ **Atenção**
> **SA não é *fire-and-forget*.** Os três parâmetros principais — T_0, α e L — interagem de forma não-trivial. O procedimento recomendado é: (1) calibrar T_0 pelo método da taxa de aceitação; (2) escolher $T_{\min}$ como fração de T_0 (tipicamente $10^{\{-3\}}$ a $10^{\{-4\}}$); (3) ajustar α para que o número total de avaliações seja compatível com o orçamento disponível.

7 Comparação HC vs. SA

Aspecto	HC com Reinício	Simulated Annealing
Ótimos locais	Reinício; sem aprendizado	Escape probabilístico via T
Custo computacional	Baixo por reinício	Moderado; depende de L e K
Parâmetros	Simples: apenas vizinhança	3 principais: T_0, α, L
Garantia teórica	Nenhuma	Convergência se $T_k \geq \frac{C}{\ln} k$
Paisagem unimodal	Excelente	Equivalente ao HC; overhead desnecessário
Paisagem multimodal	Depende de $\lvert\mathcal{B}^*\rvert$	Superior; escapa de múltiplos ótimos locais
Implementação	Trivial	Simples; requer calibração cuidadosa
Indicado para	Problemas simples, tempo real	Problemas com muitos ótimos locais, offline

Tabela 3: Hill Climbing vs. Simulated Annealing: guia de seleção.

8 Síntese e Próximos Passos

Nesta aula estabelecemos os dois primeiros algoritmos de trajetória. O Hill Climbing é o ponto de partida conceitual: simples, eficiente em problemas unimodais, mas estruturalmente incapaz de escapar de ótimos locais. O SA resolve essa limitação com elegância termodinâmica — o critério de Metropolis permite pioras controladas com probabilidade que decresce suavemente com a temperatura e com a magnitude da piora.

A parte mais rica do SA reside nos esquemas de resfriamento: a escolha entre geométrico, logarítmico e adaptativo não é arbitrária mas deve ser fundamentada na estrutura da paisagem e nas restrições de orçamento de avaliações. Aplicado ao TSP com $n = 20$ cidades, o SA demonstrou superioridade clara sobre o HC por conseguir atravessar barreiras entre bacias de atração — um movimento que exige aceitar pioras intermediárias que o HC sistematicamente recusa.

Na **Aula 4**, introduziremos a **Busca Tabu**, que substitui o mecanismo probabilístico do SA por um mecanismo determinístico baseado em **memória explícita**. Veremos que essa mudança permite um controle mais preciso sobre quais regiões do espaço são exploradas e explotadas — e como isso se reflete no desempenho sobre o TSP e o Knapsack.

9 Exercícios

Exercício

1 (Analítico — Critério de Metropolis). Para o TSP com distâncias em km, calcule a probabilidade de aceitar uma piora de $\Delta f = 50$ km para temperaturas $T \in \{5, 20, 80, 200\}$ km. Organize os resultados em tabela e discuta: para qual temperatura o SA começa a se comportar como busca

aleatória? Para qual se comporta como HC? Como esses limiares mudam se a escala de f for alterada (e.g., distâncias em metros em vez de km)?

Exercício

2 (Esquemas de Resfriamento). Implemente em Python os quatro esquemas da Tabela 1 e plote, em um mesmo gráfico, a evolução de T_k em função da iteração k, para os parâmetros: $T_0 = 200$, $T_f = 0.1$, $K = 500$ (linear e logarítmico), $\alpha = 0.95$ (geométrico). Para o adaptativo, simule $\sigma_{\Delta f} = 40$ constante. Discuta as diferenças no tempo gasto em cada faixa de temperatura e o que isso implica para o balanço exploração/explotação ao longo da execução.

Exercício

3 (Calibração de T_0) - *1,0 ponto.* Gere uma instância do TSP com $n = 30$ cidades (coordenadas uniformes em $[0, 100]^2$, semente de sua escolha, registrada). Implemente uma rotina de calibração automática: execute 1000 movimentos 2-opt aleatórios, calcule $\overline{\Delta f^+}$ e determine T_0 para taxas de aceitação iniciais $\chi_0 \in \{0.60, 0.75, 0.90\}$. Execute o SA com cada valor de T_0 (30 execuções independentes cada, $\alpha = 0.95$) e compare as curvas de convergência. Qual valor de χ_0 produz melhor resultado final? Justifique à luz da teoria.

Exercício

4 (Implementação Completa — SA para TSP) - *1,0 ponto.* Implemente o SA com operador 2-opt para o TSP com $n = 30$ cidades (mesma instância do Exercício 3). Compare com o HC com reinício (100 reinícios, mesmo orçamento total de avaliações). Execute 30 rodadas independentes de cada algoritmo. Reporte: distância média, desvio padrão, melhor resultado encontrado, e número total de avaliações de f. Aplique o teste de Wilcoxon para verificar a significância estatística da diferença. Plote a melhor rota encontrada por cada método.

Leituras recomendadas:

• Kirkpatrick, S., Gelatt, C. D., Vecchi, M. P. Optimization by simulated annealing. **Science**, 220(4598), 671–680, 1983. *(artigo original — leitura obrigatória)*
• Aarts, E., Korst, J. **Simulated Annealing and Boltzmann Machines**. Wiley, 1989. Caps. 2–3.
• Johnson, D. S. et al. Optimization by simulated annealing: An experimental evaluation. **Operations Research**, 37(6), 865–892, 1989. *(referência para calibração de parâmetros)*
• Applegate, D. L. et al. **The Traveling Salesman Problem: A Computational Study**. Princeton University Press, 2006. Cap. 1.
• Lin, S., Kernighan, B. W. An effective heuristic algorithm for the traveling-salesman problem. **Operations Research**, 21(2), 498–516, 1973. *(operador 2-opt e variantes)*